Microservices Architecture Evolution

A Practitioner's Solution Guide — From Monolith to Event-Driven,
Serverless & Cloud-Native Microservices (Java / Spring Boot)

DEFINITION · HANDS-ON · REAL-TIME USAGE · ARCHITECTURE DIAGRAMS

Table of Contents

1. Monolithic Architecture
2. Modular Monolith Architecture
3. Microservices & Event-Driven Microservices
4. Serverless Architecture (Millions of Requests)
5. Microservices Decomposition
6. Microservices Communications
7. Microservices Data Management (CQRS + Event Sourcing)
8. Microservices Distributed Caching
9. Microservices Deployments (CI/CD, Canary, Blue-Green)
10. 3-Tier Architecture
11. API Gateway
12. Backend For Frontend (BFF)
13. Service Aggregator & Registry Patterns
14. Message Broker
15. Database Sharding with Cassandra
16. Event-Driven Microservices Architecture (Choreographed Saga)
17. Containers, Orchestration & CI/CD

01 Monolithic Architecture

DEFINITION

- A single unified codebase containing UI, business logic, and data access, packaged and deployed as one artifact (JAR/WAR).
- Horizontal scaling means duplicating the entire application, not individual features.
- Simple to develop, test, and deploy for small teams and early-stage products.

HANDS-ON

Spring Boot Monolith (Order Service with Redis Cache)

```
@Service
public class OrderService {
    private final OrderRepository orderRepository;

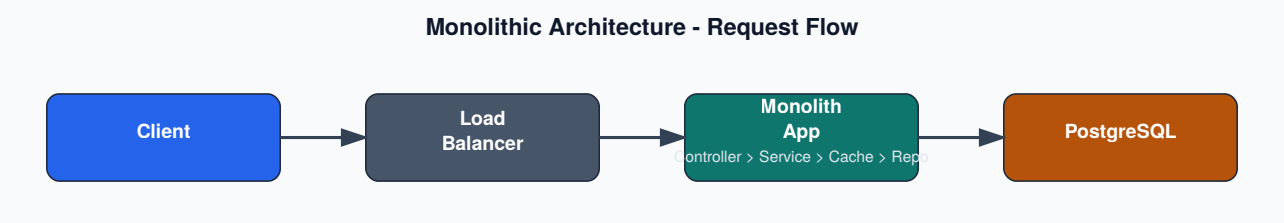
    @Cacheable(value = "orders", key = "#id", unless = "#result == null")
    public Order getOrderById(String id) {
        return orderRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Order not found"));
    }

    @CacheEvict(value = "orders", key = "#order.id")
    public Order createOrder(Order order) {
        order.setStatus(OrderStatus.PENDING);
        return orderRepository.save(order);
    }
}
```

REAL-TIME USAGE

Organization	How it is applied
Startups / MVPs	Speed of delivery matters more than horizontal scalability.
Internal back-office tools	Low concurrency, simple CRUD, low operational cost.
Latency-critical domains	In-process calls avoid network hops entirely.
Legacy enterprise systems	Modernized gradually via the Strangler Fig pattern.

ARCHITECTURAL FLOW DIAGRAM



1. Load balancer routes the request to a healthy monolith instance.
2. Controller deserializes the request and calls the Service layer.
3. Service checks Redis cache (@Cacheable) before hitting PostgreSQL.
4. On a cache miss, the Repository queries PostgreSQL via JPA.
5. Response is serialized back to JSON and returned to the client.

Trade-offs / What this evolves into next

- Tight coupling: one module's change forces a full redeploy.

- Scaling inefficiency: must scale the whole JAR even for one hot module.
- Single point of failure and technology lock-in.

02 Modular Monolith Architecture

DEFINITION

- A single deployable unit, but the codebase is split into independent, highly cohesive modules (DDD bounded contexts).
- Modules communicate via in-process interfaces or an internal event bus, never by reaching into each other's repositories.
- Data is isolated per-module using separate schemas within the same database cluster.

HANDS-ON

Module Interface + Internal Event Bus

```
public interface InventoryFacade {
    boolean reserveStock(String productId, int quantity);
}

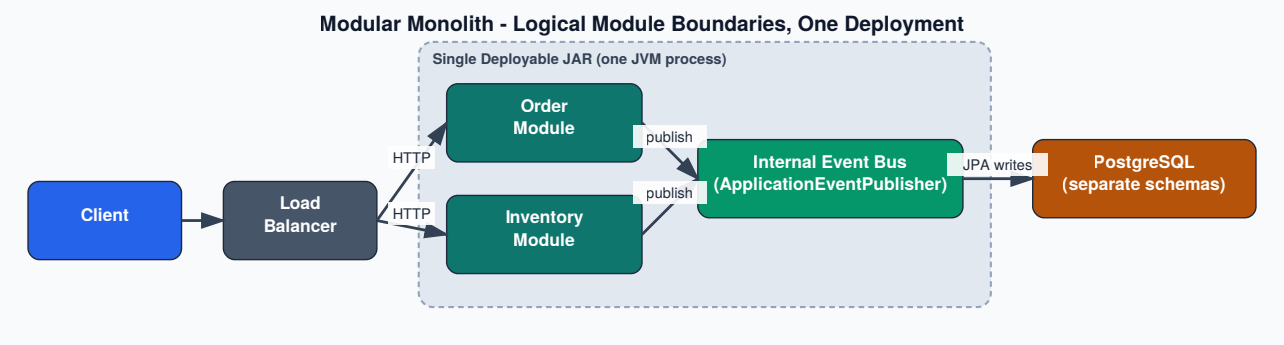
@Service
public class OrderService {
    private final InventoryFacade inventoryFacade;      // sync, in-JVM call
    private final ApplicationEventPublisher eventPublisher;

    @Transactional
    public Order createOrder(Order order) {
        Order saved = orderRepository.save(order);
        if (!inventoryFacade.reserveStock(order.getProductId(), order.getQuantity()))
            throw new RuntimeException("Out of stock");
        eventPublisher.publishEvent(new OrderPlacedEvent(this, saved.getId()));
        return saved;
    }
}
```

REAL-TIME USAGE

Organization	How it is applied
Shopify	Large-scale modular monolith with guild-owned modules.
Atlassian (Jira)	500+ engineers share one deployable with strict module ownership.
Fintech platforms	ACID transactions across modules without distributed-transaction overhead.
Scale-up SaaS (5->50 engineers)	Dev velocity of microservices, ops simplicity of a monolith.

ARCHITECTURAL FLOW DIAGRAM



1. Order module persists the order and calls Inventory module synchronously (in-process, ~1ms).
2. Both writes happen inside one @Transactional boundary spanning two schemas (ACID).
3. After commit, an OrderPlacedEvent is published to the internal Spring event bus.
4. Async @EventListener handlers react without blocking the HTTP response.

Trade-offs / What this evolves into next

- Deployment coupling remains: one module's bug still forces a full redeploy.
- Cannot scale or independently release modules.
- Still limited to a single technology stack (JVM).

03 Microservices & Event-Driven Microservices

DEFINITION

- Independently deployable services, each owning its own database (database-per-service).
- Event-driven communication: services publish immutable events to Kafka instead of calling each other synchronously.
- Choreography over orchestration: services react autonomously; consistency is eventual, not distributed-transactional.

HANDS-ON

Order Service Producer + Inventory Consumer

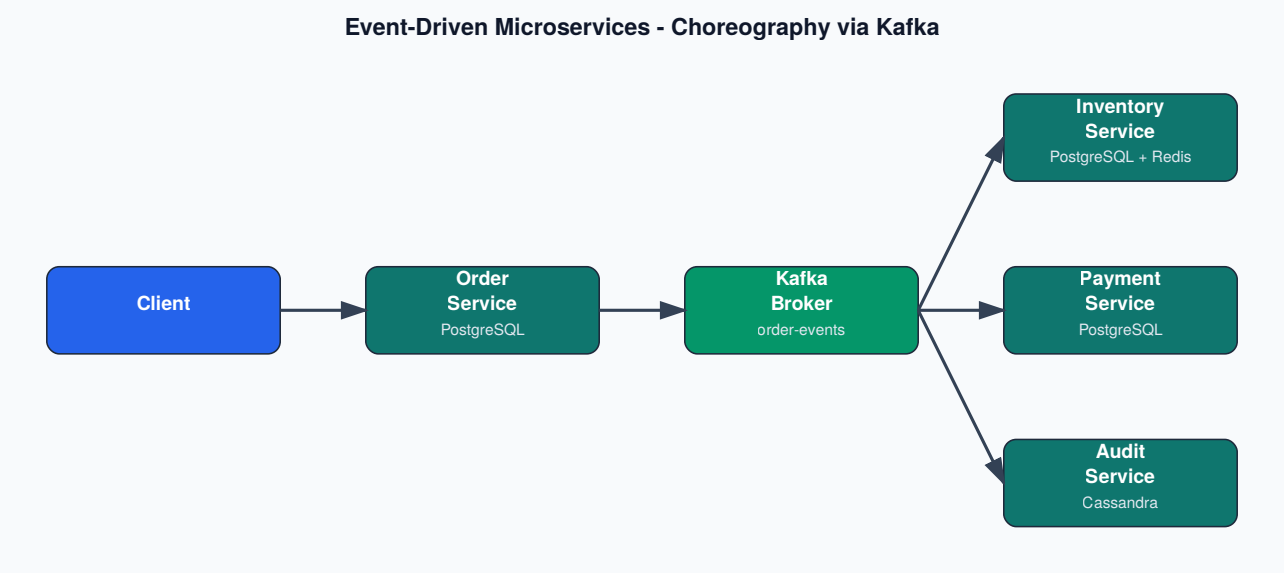
```
// Producer (Order Service)
kafkaTemplate.send("order-events", saved.getId(), event);

// Consumer (Inventory Service)
@KafkaListener(topics = "order-events")
public void handleOrderCreated(OrderCreatedEvent event) {
    if (inventoryRepository.reserve(event.getProductId(), event.getQuantity()) > 0) {
        kafkaTemplate.send("inventory-events", new InventoryReservedEvent(event.getOrderId()));
    } else {
        kafkaTemplate.send("inventory-events", new InventoryFailedEvent(event.getOrderId()));
    }
}
```

REAL-TIME USAGE

Organization	How it is applied
Netflix	700+ microservices; encoding pipeline triggered by upload events.
Uber	Millions of trip events/sec drive dispatch, pricing, and notifications.
E-commerce checkout	Fraud detection and email run asynchronously after 202 Accepted.
Financial settlement	Trades ingested via Kafka; settlement/risk/reporting run isolated.

ARCHITECTURAL FLOW DIAGRAM



1. Order Service saves PENDING order to its own PostgreSQL and returns 202 Accepted immediately.
2. OrderCreatedEvent is published to Kafka; the client does not wait for downstream steps.

3. Inventory, Payment, and Audit services consume the same event independently and in parallel.
4. Audit Service persists every event to Cassandra for a high-throughput, immutable trail.

Trade-offs / What this evolves into next

- Debugging spans multiple JVMs and databases without correlation IDs.
- Network hops add latency; requires circuit breakers for resilience.
- Risk of losing events on producer crash unless the Outbox pattern is used.

04 Serverless Architecture (Millions of Requests)

DEFINITION

- Function-as-a-Service (FaaS): stateless functions triggered by HTTP or events, scaling to zero and back automatically.
- Managed services replace self-hosted infrastructure: DynamoDB/Aurora Serverless for data, SQS/EventBridge for messaging, ElastiCache Serverless for caching.
- Java cold starts are mitigated with Provisioned Concurrency or GraalVM native images.

HANDS-ON

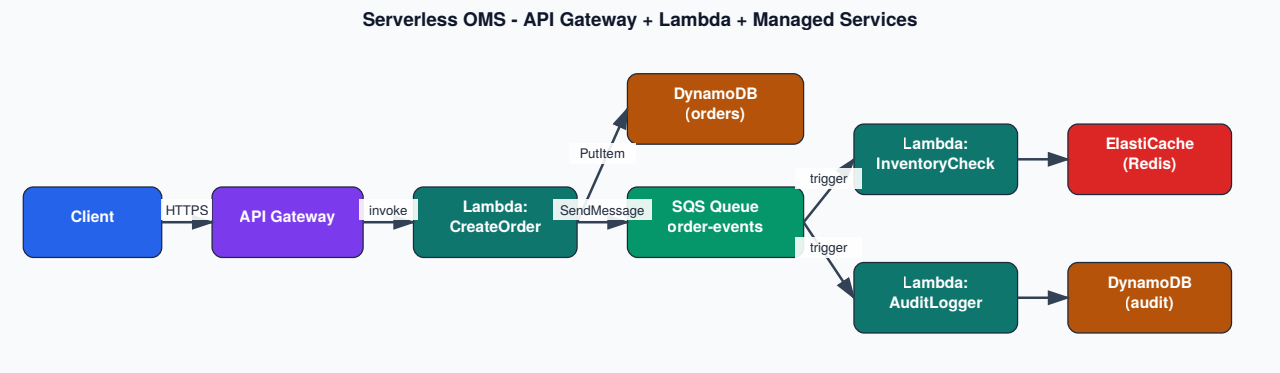
Spring Cloud Function Lambda Handler

```
@Component
public class OrderCreateFunction implements Function<CreateOrderRequest, OrderResponse> {
    @Override
    public OrderResponse apply(CreateOrderRequest request) {
        Order order = Order.builder().orderId(UUID.randomUUID().toString())
                .status("PENDING").build();
        orderTable.putItem(order).join();                // DynamoDB, async
        sqsClient.sendMessage(toSendMessageRequest(order)).join(); // fan-out
        return new OrderResponse(order.getOrderId(), "PENDING");
    }
}
```

REAL-TIME USAGE

Organization	How it is applied
Netflix encoding pipeline	Step Functions orchestrate 100+ Lambdas per upload.
Capital One	Card transactions processed in ~200ms, fully serverless.
Taco Bell	Lunch-rush order API scales instantly without server sizing.
IoT fleets	Millions of telemetry events ingested via Kinesis + Lambda.

ARCHITECTURAL FLOW DIAGRAM



1. API Gateway throttles and forwards requests to the CreateOrder Lambda.
2. Lambda writes to DynamoDB (On-Demand capacity absorbs 1M writes/sec) and enqueues to SQS.
3. SQS decouples the write path from Inventory/Audit processing, applying natural backpressure.
4. Lambda aliases with traffic weights enable Canary and instant Blue-Green rollback.

Trade-offs / What this evolves into next

- Cold starts (mitigated by Provisioned Concurrency / SnapStart).

- No local state; everything externalizes to DynamoDB/Redis.
- Vendor lock-in to managed services (SQS, EventBridge, DynamoDB).

05 Microservices Decomposition

DEFINITION

- The disciplined process of extracting bounded contexts out of a monolith without downtime, guided by Domain-Driven Design.
- Database-per-service is the non-negotiable end state; data is mirrored via CDC (Debezium) during transition.
- The Strangler Fig pattern gradually reroutes traffic from the monolith to the new service.

HANDS-ON

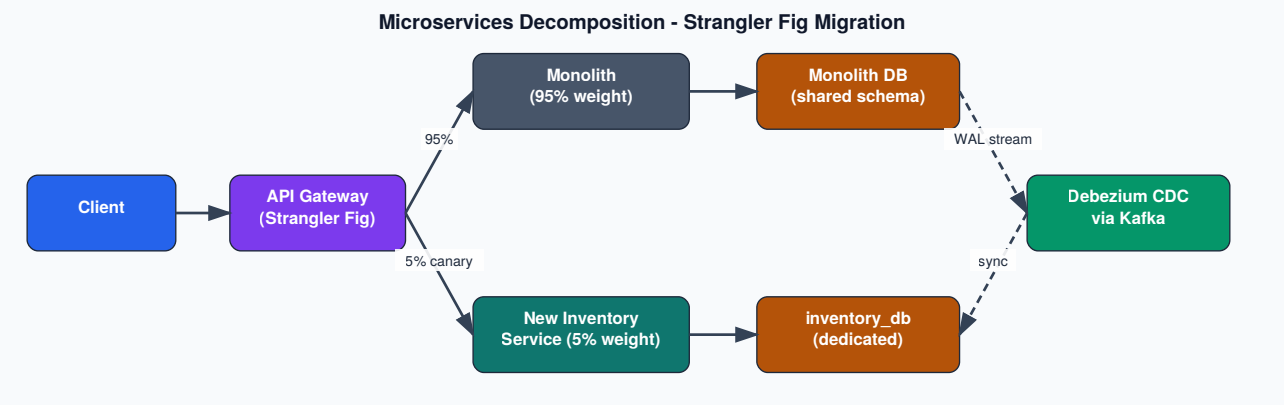
Weighted Gateway Routing (Canary Extraction)

```
spring:
  cloud:
    gateway:
      routes:
        - id: inventory-canary
          uri: lb://inventory-service
          predicates: [Path=/api/inventory/**]
          filters: [{name: Weight, args: {group: inv, weight: 5}}]
        - id: inventory-legacy
          uri: lb://oms-monolith
          predicates: [Path=/api/inventory/**]
          filters: [{name: Weight, args: {group: inv, weight: 95}}]
```

REAL-TIME USAGE

Organization	How it is applied
Amazon (2000s)	The API Mandate forced all teams onto internal service APIs.
Uber Payments	Repository pattern swapped for HTTP calls over 6 months, zero downtime.
ING Bank	Two-phase decomposition: extract writes via CDC, then cut over reads.

ARCHITECTURAL FLOW DIAGRAM



1. Debezium streams the monolith's PostgreSQL WAL into the new inventory_db (out-of-band).
2. The Gateway sends a small weighted slice of traffic to the new service (Canary).
3. Feature flags progressively move writes from the local Java call to the Feign/HTTP client.
4. Once parity is verified, weight moves to 100% and the old schema/module is deleted.

Trade-offs / What this evolves into next

- Requires careful data-parity verification during the dual-write window.

- Temporary added complexity (CDC pipeline, feature flags) during migration.

06 Microservices Communications

DEFINITION

- Synchronous: RestTemplate (legacy), OpenFeign (declarative REST), gRPC (binary, low-latency RPC).
- Asynchronous: Kafka for eventual consistency and fan-out.
- API Gateway sits at the edge for routing, rate limiting, and cross-cutting concerns.

HANDS-ON

gRPC Client + Kafka Producer Side by Side

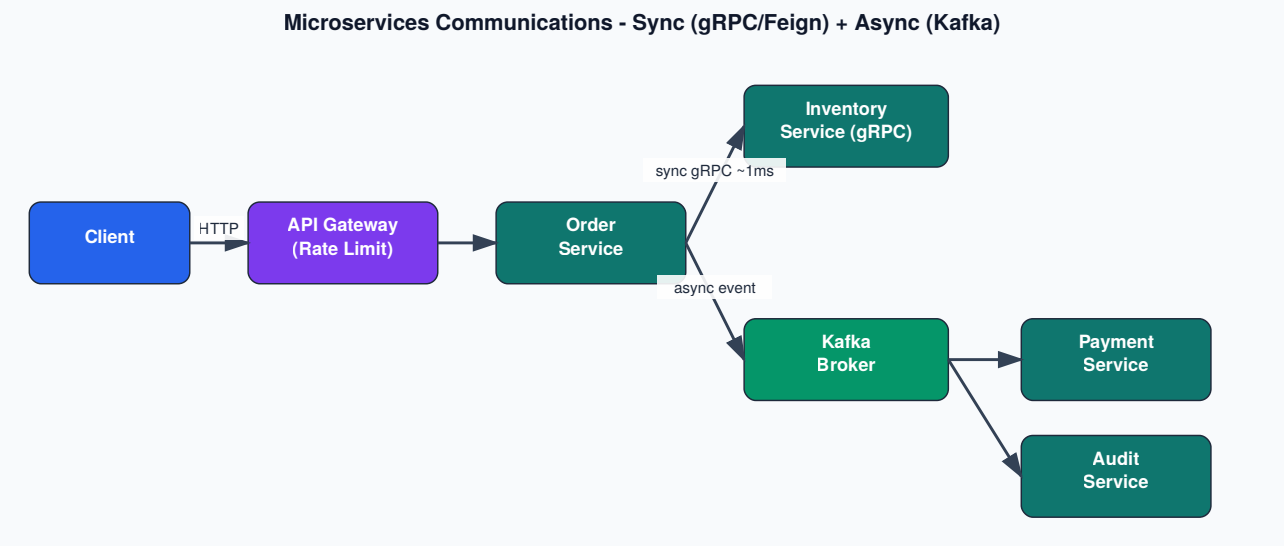
```
// Low-latency sync call for the critical path
ReserveResponse response = blockingStub.reserveStock(request); // ~1-2ms

// Non-critical async fan-out after the HTTP response is already sent
kafkaTemplate.send("order-events", order.getId(), event);
```

REAL-TIME USAGE

Organization	How it is applied
Uber	gRPC for driver matching; Kafka for trip lifecycle events.
LinkedIn	Kafka streams millions of profile updates to search and newsfeed.
Etsy	API Gateway aggregates listing, shipping, and user data in parallel.

ARCHITECTURAL FLOW DIAGRAM



1. Gateway authenticates and rate-limits the request before it reaches any service.
2. Order Service calls Inventory synchronously over gRPC for the critical stock check (~1-2ms).
3. After committing locally, Order Service publishes an event to Kafka for non-blocking side effects.
4. Payment and Audit consume the event independently, off the request's critical path.

Trade-offs / What this evolves into next

- Choosing sync vs async per call is a design decision, not a default.
- gRPC requires a proxy for browser clients; Kafka adds operational overhead.

07 Microservices Data Management (CQRS + Event Sourcing)

DEFINITION

- CQRS separates the write model (Command, PostgreSQL, ACID) from the read model (Query, Cassandra, denormalized).
- Event Sourcing stores every state change as an immutable event, which a Projection Service replays into the read model.
- Cross-service consistency is achieved via the Saga pattern (choreography), never via 2PC.

HANDS-ON

Command Write + Projection to Cassandra

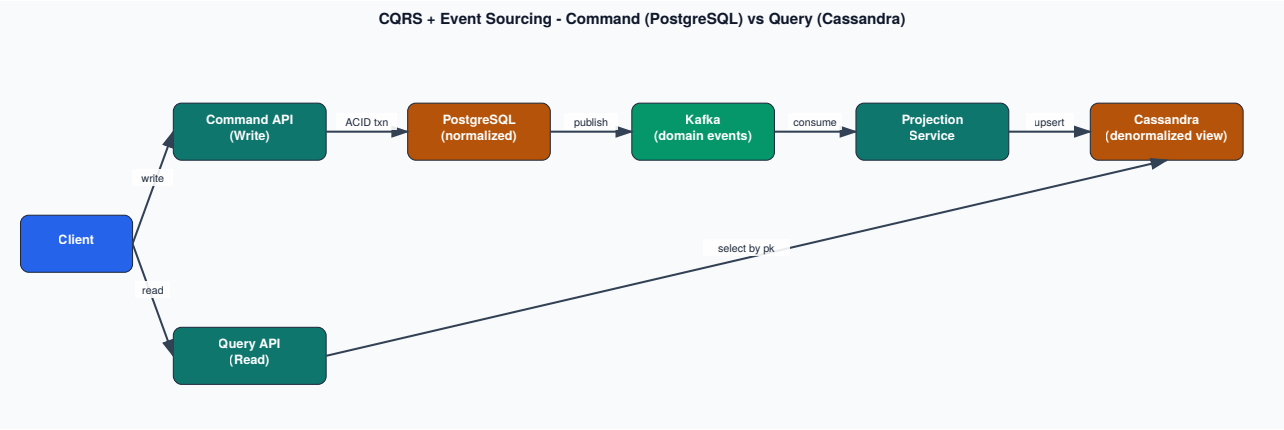
```
@Transactional
public Order createOrder(CreateOrderCommand cmd) {
    Order saved = orderRepository.save(toOrder(cmd));          // PostgreSQL, ACID
    kafkaTemplate.send("order-events", new OrderCreatedEvent(saved));
    return saved;
}

@KafkaListener(topics = "order-events")
public void onOrderEvent(OrderCreatedEvent e) {
    cassandraRepo.save(buildDenormalizedView(e));               // Cassandra, fast reads
}
```

REAL-TIME USAGE

Organization	How it is applied
Goldman Sachs	Trades event-sourced; dashboards read from denormalized Elasticsearch/Cassandra views.
Zalando	Order lifecycle event-sourced; tracking UI reads a dedicated Cassandra view.
Uber	Trip lifecycle event-sourced; driver app reads from a low-latency view store.

ARCHITECTURAL FLOW DIAGRAM



1. Command API validates and writes the order to PostgreSQL inside an ACID transaction.
2. It publishes a domain event to Kafka after the local commit succeeds.
3. The Projection Service consumes the event, enriches it, and upserts a denormalized row into Cassandra.
4. The Query API reads directly from Cassandra by partition key, independent of the write path.

Trade-offs / What this evolves into next

- ~50-100ms eventual-consistency lag between write and read-model update.

- Two databases to operate and reconcile instead of one.

08 Microservices Distributed Caching

DEFINITION

- L1 (Caffeine): in-process, per-pod, microsecond latency, not shared across instances.
- L2 (Redis): centralized, shared across all pods, millisecond latency.
- @Cacheable (read-through), @CachePut (write-through), @CacheEvict (invalidate) are Spring's core annotations.
- Eviction policies: LRU (general-purpose), LFU (skewed access), TTL (time-sensitive data).

HANDS-ON

Multi-Tier Read with Manual Fallback

```
public Product getProductById(String id) {
    Product p = caffeineCache.get(id, Product.class);
    if (p != null) return p;                    // L1 hit, ~microseconds

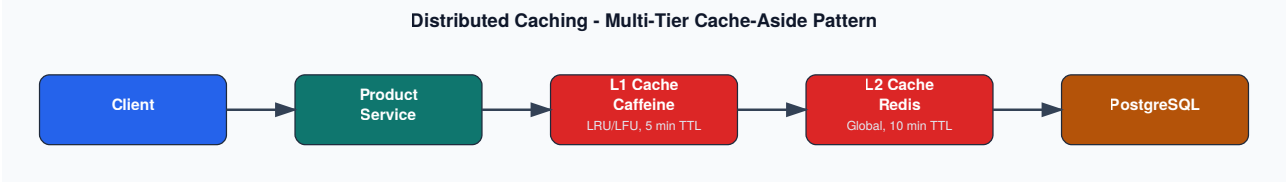
    p = redisCache.get(id, Product.class);
    if (p != null) { caffeineCache.put(id, p); return p; }  // L2 hit, backfill L1

    p = productRepository.findById(id).orElseThrow(); // DB fallback
    redisCache.put(id, p); caffeineCache.put(id, p);
    return p;
}
```

REAL-TIME USAGE

Organization	How it is applied
Walmart product catalog	Caffeine LRU locally + Redis globally, ~10-minute TTL.
Netflix API rate limiter	Redis TTL-based sliding-window counters.
Banking JWT blacklist	Caffeine LFU keeps frequently-abused tokens hot.

ARCHITECTURAL FLOW DIAGRAM



1. Request checks Caffeine (L1) first; a hit returns in low single-digit microseconds.
2. On an L1 miss, Redis (L2) is checked; a hit backfills L1 for the next request on this pod.
3. On a full miss, PostgreSQL is queried and both cache tiers are populated.
4. Writes use @CachePut to refresh Redis and manually evict the local Caffeine entry.

Trade-offs / What this evolves into next

- Cache stampede on a hot key expiring under load (mitigate with refreshAfterWrite).
- Eventual consistency window between a write and other pods' local L1 caches.

09 Microservices Deployments (CI/CD, Canary, Blue-Green)

DEFINITION

- CI/CD automates build, test, containerization, and deployment on every merge.
- Blue-Green: two identical environments; the router flips instantly, giving a near-zero-time rollback.
- Canary: a small percentage of traffic hits the new version first, ramping up only if metrics stay healthy.

HANDS-ON

Blue-Green Cutover via Service Selector Patch

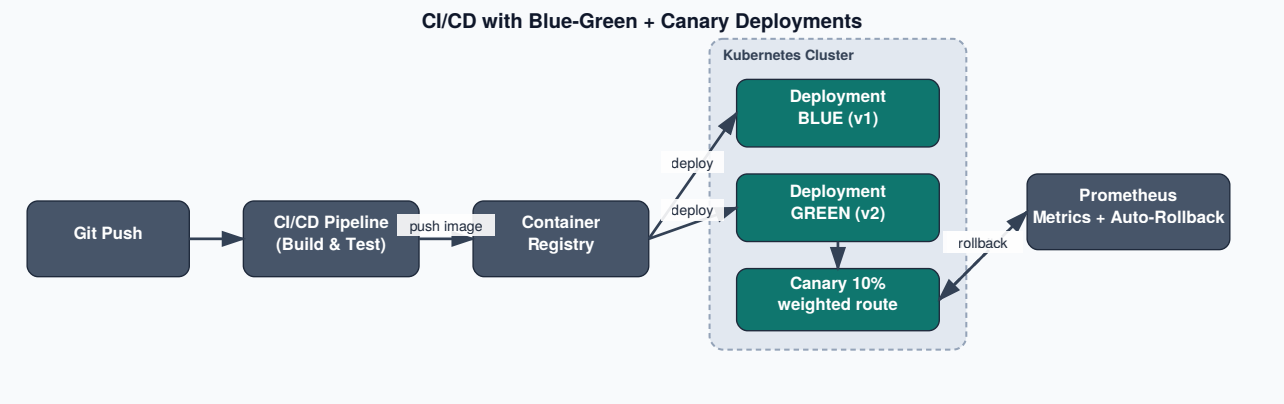
```
# Instant cutover
kubectl patch service inventory-service-svc \
  -p '{"spec":{"selector":{"version":"green"}}}'

# Instant rollback (<1s)
kubectl patch service inventory-service-svc \
  -p '{"spec":{"selector":{"version":"blue"}}}'
```

REAL-TIME USAGE

Organization	How it is applied
Netflix (Spinnaker)	Blue-Green with automated integration tests before cutover.
Uber (Istio Canary)	1% of riders test new matching algorithms before full rollout.
Etsy	300+ deploys/day using a Canary + Blue-Green hybrid.

ARCHITECTURAL FLOW DIAGRAM



1. CI pipeline builds, tests, and pushes an immutable image tag to the registry.
2. The new version deploys as GREEN and passes automated smoke tests before receiving any traffic.
3. A weighted VirtualService (Istio) ramps Canary traffic from 10% upward while Prometheus watches error rate.
4. Any error-rate spike triggers an automatic rollback to the previous stable version.

Trade-offs / What this evolves into next

- Requires investment in observability to make rollback decisions automatically.
- Blue-Green doubles resource usage during the cutover window.

10 3-Tier Architecture

DEFINITION

- Physically separates Presentation (UI), Application (business logic), and Data (database) onto distinct tiers/servers.
- Distinct from 'layers': layers are logical divisions inside one process; tiers are physical/network divisions.
- Network policy typically forbids the Presentation tier from reaching the Data tier directly.

HANDS-ON

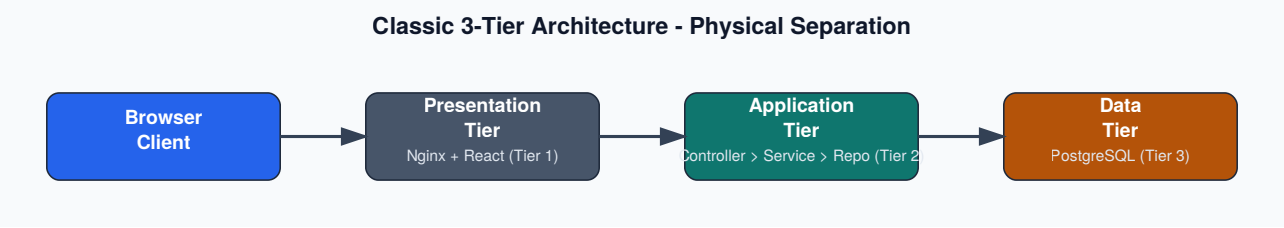
Docker Compose - Physical Tier Separation

```
services:
  ui:   { image: nginx:alpine, networks: [frontend-network] }
  api:  { build: ./api, networks: [frontend-network, backend-network] }
  db:   { image: postgres:15-alpine, networks: [backend-network] }
```

REAL-TIME USAGE

Organization	How it is applied
Banking core systems	Mainframe (Tier 3) isolated behind an application server (Tier 2).
Oracle EBS / SAP	Web dispatcher never talks to the DB directly; always via the app tier.
Healthcare (HIPAA)	Presentation in a DMZ; data tier fully air-gapped from the internet.

ARCHITECTURAL FLOW DIAGRAM



1. Nginx (Tier 1) serves the static SPA and proxies API calls to the Application Tier.
2. The Application Tier's Controller > Service > Repository layers apply business rules.
3. Only the Application Tier has network access to PostgreSQL (Tier 3).
4. Docker network isolation enforces this physically: the UI network never joins the backend network.

Trade-offs / What this evolves into next

- If the Application Tier grows unchecked, it collapses back into a monolith.
- Scaling is coarse-grained: the whole Application Tier scales together.

11 API Gateway

DEFINITION

- A reverse proxy and single entry point that applies routing, authentication, rate limiting, and circuit breaking at the edge.
- Shields clients from internal topology changes and protects internal services from unauthenticated or excessive traffic.
- Can aggregate multiple backend calls into a single client-facing response.

HANDS-ON

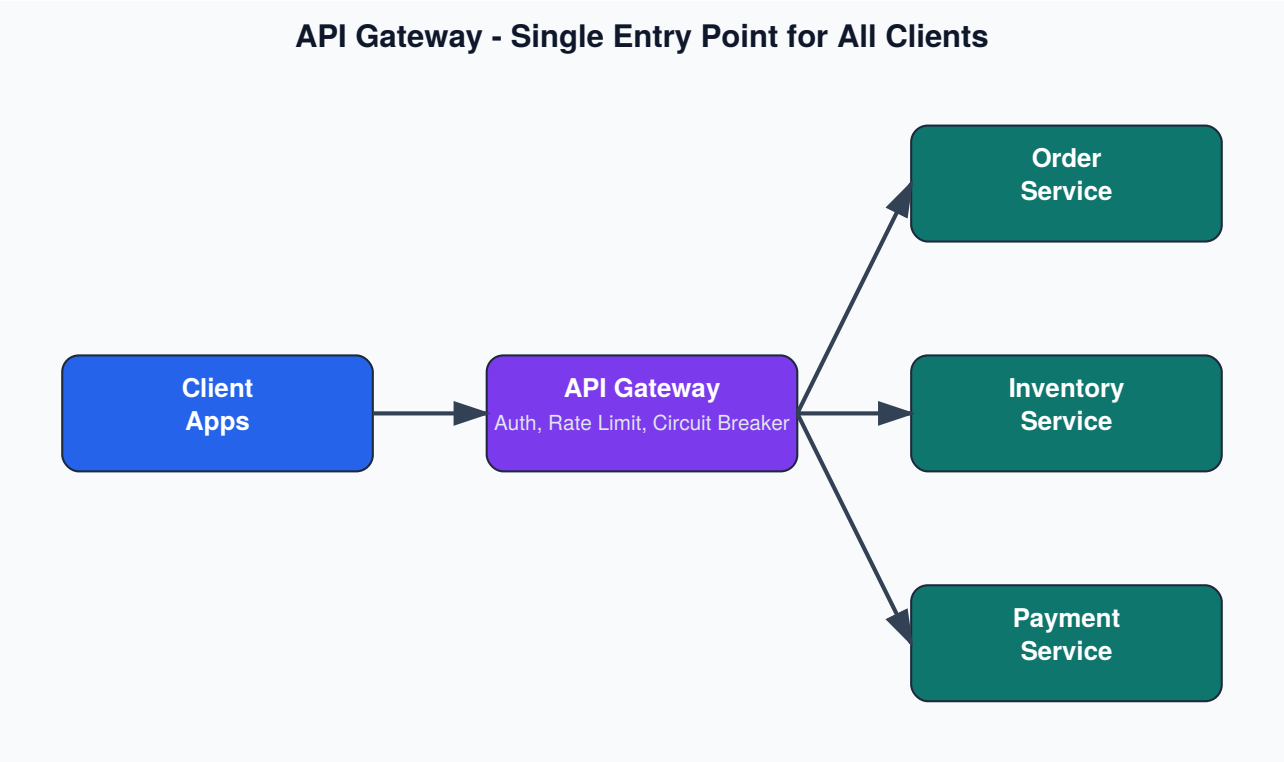
Spring Cloud Gateway - JWT Auth + Rate Limit + Circuit Breaker

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-route
          uri: lb://order-service
          predicates: [Path=/api/orders/**]
          filters:
            - name: CircuitBreaker
              args: { name: orderCB, fallbackUri: forward:/fallback/orders }
            - name: RequestRateLimiter
              args: { redis-rate-limiter.replenishRate: 10, redis-rate-limiter.burstCapacity: 20 }
```

REAL-TIME USAGE

Organization	How it is applied
Netflix (Zuul)	Authenticates devices and geo-routes over 2M req/sec at peak.
Amazon API Gateway	Translates REST into Lambda invocations, caches at CloudFront.
JPMorgan	Enforces mutual TLS at the edge before requests reach payment services.

ARCHITECTURAL FLOW DIAGRAM



1. The JWT filter validates the token and injects a lightweight user-id header downstream.
2. The rate limiter checks Redis for the caller's token bucket before allowing the request through.
3. The circuit breaker short-circuits calls to an unhealthy service, returning a fallback instantly.
4. Eureka-based load balancing (lb://service-name) picks a healthy instance for the final hop.

Trade-offs / What this evolves into next

- Must stay stateless and lightweight; heavy aggregation belongs in a BFF, not the Gateway.
- Becomes a single point of failure if not deployed with redundancy.

12 Backend For Frontend (BFF)

DEFINITION

- A dedicated backend per client type (Mobile, Web, Partner), each shaping and aggregating data differently.
- Mobile BFF returns minimal payloads for low-bandwidth, battery-constrained clients.
- Web BFF returns rich, denormalized payloads suited to high-bandwidth desktop rendering.

HANDS-ON

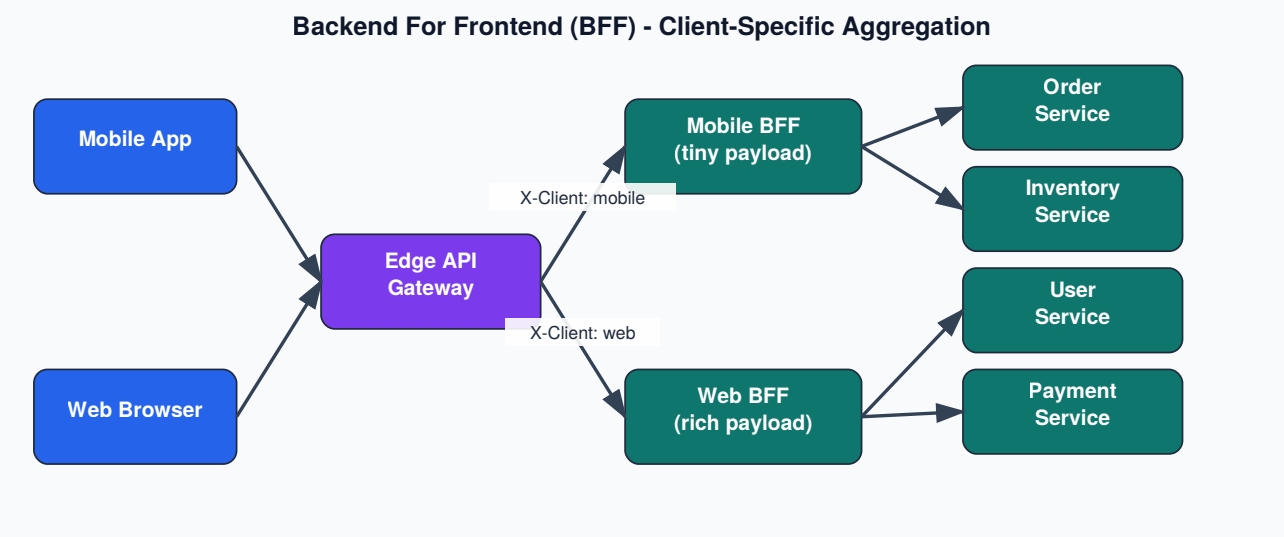
Mobile BFF - Parallel Aggregation with a Tight Timeout

```
return Mono.zip(orderMono, inventoryMono, userMono)
    .map(t -> new MobileOrderResponse(
        t.getT1().getId(), t.getT1().getStatus(), t.getT1().getTotal(),
        t.getT2().getStockLevel(), t.getT3().getFirstName()))
    .timeout(Duration.ofMillis(500));
```

REAL-TIME USAGE

Organization	How it is applied
Spotify	Mobile BFF returns 20 tracks; Web BFF returns the full catalog and art.
Netflix	TV BFF optimizes for remote-control interaction; Mobile BFF saves data roaming.
Etsy	Seller Web BFF serves financial charts; Buyer Mobile BFF serves images and price only.

ARCHITECTURAL FLOW DIAGRAM



1. The Edge Gateway routes by client-type header to the Mobile BFF or Web BFF.
2. Each BFF fires parallel non-blocking calls to only the core services it actually needs.
3. The Mobile BFF trims the response to a handful of fields; the Web BFF keeps the full graph.
4. Business logic never lives in the BFF; it stays inside the core services.

Trade-offs / What this evolves into next

- More services to operate; must avoid duplicating business rules across BFFs.

13 Service Aggregator & Registry Patterns

DEFINITION

- Service Registry (Eureka/Consul) is the dynamic phonebook: services register on startup and are discovered by name.
- Service Aggregator is an internal orchestrator that composes data from multiple services into one response.
- Unlike a BFF, an Aggregator is client-agnostic and used for internal backend-to-backend composition.

HANDS-ON

Aggregator Using Discovery-Aware WebClient

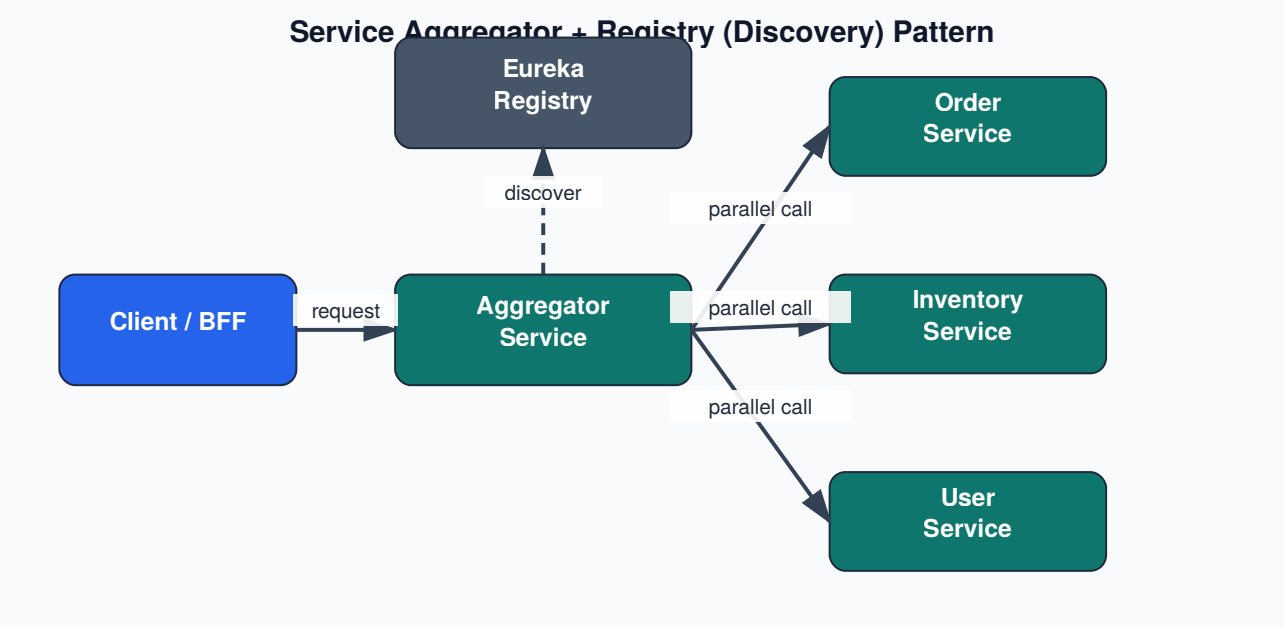
```
Mono<Order> orderMono = webClient.get()
    .uri("lb://order-service/api/orders/{id}", orderId).retrieve().bodyToMono(Order.class);
Mono<Inventory> invMono = webClient.get()
    .uri("lb://inventory-service/api/inventory/order/{id}", orderId).retrieve().bodyToMono(Inventory.class);

return Mono.zip(orderMono, invMono).map(t -> new OrderDetailsResponse(t.getT1(), t.getT2()));
```

REAL-TIME USAGE

Organization	How it is applied
Netflix	Playback Aggregator composes media metadata, preferences, and DRM licenses.
Amazon product page	Aggregator merges product, pricing, reviews, and inventory in parallel.
Bloomberg	Portfolio Aggregator combines market data, FX rates, and holdings via Consul discovery.

ARCHITECTURAL FLOW DIAGRAM



1. The Aggregator resolves service instances dynamically through the Eureka registry (lb:// URIs).
2. It fires parallel calls to Order, Inventory, and User services using non-blocking WebClient.
3. Mono.zip waits for all responses (bounded by a timeout) and composes a single unified DTO.
4. If a downstream instance dies, Eureka evicts it after missed heartbeats, keeping routing healthy.

Trade-offs / What this evolves into next

- Must implement onErrorResume fallbacks per call to avoid one failure sinking the whole response.

14 Message Broker

DEFINITION

- Middleware that decouples producers from consumers, buffering and reliably routing messages.
- Log-based brokers (Kafka) retain and allow replay; queue-based brokers (RabbitMQ, SQS) delete on acknowledgment.
- Idempotent producers, manual offset commits, and Dead Letter Queues are the three pillars of reliability.

HANDS-ON

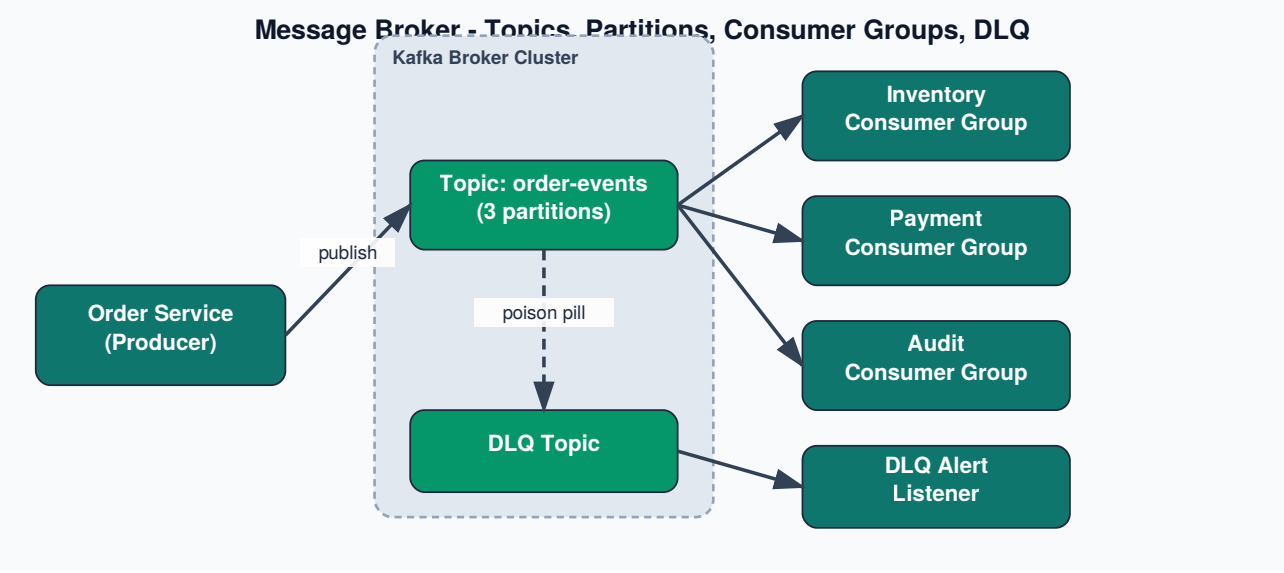
Manual-Ack Consumer with Dead Letter Queue

```
@KafkaListener(topics = "order-events", groupId = "inventory-group")
public void consume(ConsumerRecord<String, OrderCreatedEvent> record, Acknowledgment ack) {
    try {
        inventoryService.reserveStock(record.value());
        ack.acknowledge();
    } catch (NonRetryableException e) {
        kafkaTemplate.send("order-events-dlq", record.key(), record.value());
        ack.acknowledge();          // unblock the partition
    }
}
```

REAL-TIME USAGE

Organization	How it is applied
LinkedIn	7+ trillion Kafka messages/day power the newsfeed and recommendations.
Slack	Kafka streams typing indicators and presence to 10M+ concurrent users.
Walmart	Azure Service Bus queues absorb Black-Friday checkout spikes without lost orders.

ARCHITECTURAL FLOW DIAGRAM



1. The producer writes with acks=all and idempotence enabled, guaranteeing exactly-once appends.
2. Multiple consumer groups (Inventory, Payment, Audit) each receive their own copy of every event.
3. A consumer only commits its offset after successfully processing the message (manual ack).
4. Poison-pill messages are rerouted to a DLQ so they never block the rest of the partition.

Trade-offs / What this evolves into next

- Consumers must be idempotent since At-Least-Once delivery allows duplicates.
- Consumer lag must be actively monitored to catch services falling behind.

15 Database Sharding with Cassandra

DEFINITION

- Horizontal partitioning of data across nodes using a Partition Key, distributed via consistent hashing on a token ring.
- Clustering Keys define the sort order within a partition (e.g., most recent order first).
- Data modeling is query-first: design tables around your read patterns, not normalized relations.

HANDS-ON

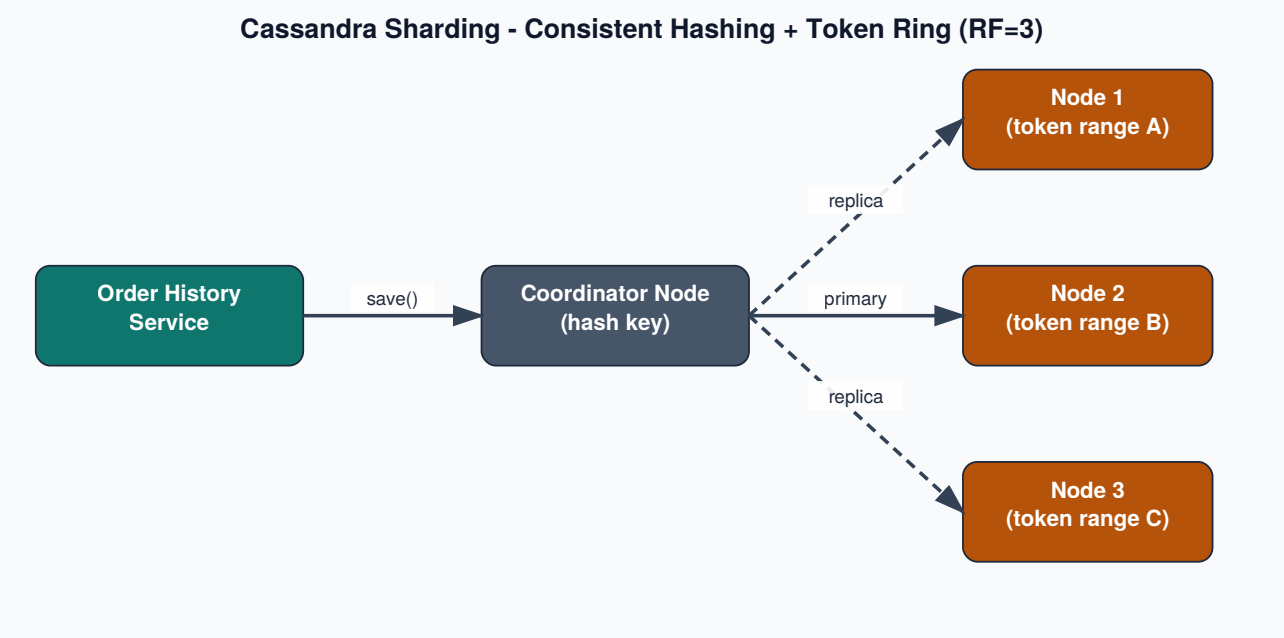
Query-First Table Design

```
CREATE TABLE orders_by_customer (
    customer_id text,
    order_id text,
    order_date timestamp,
    total_amount double,
    PRIMARY KEY (customer_id, order_id, order_date)
) WITH CLUSTERING ORDER BY (order_date DESC);
```

REAL-TIME USAGE

Organization	How it is applied
Netflix	Viewing history partitioned by user_id across hundreds of billions of rows.
Discord	Messages partitioned by channel_id for fast infinite-scroll retrieval.
Uber	Trip history partitioned by driver_id/trip_date for append-heavy writes.

ARCHITECTURAL FLOW DIAGRAM



1. The driver hashes the partition key (Murmur3) to compute a token value.
2. The token maps to a primary node on the ring, which becomes the write coordinator.
3. The coordinator writes locally and replicates to the next RF-1 nodes on the ring.
4. With consistency-level QUORUM, the client gets an ack once a majority of replicas confirm.

Trade-offs / What this evolves into next

- A poorly chosen (low-cardinality) partition key creates hot nodes.
- Partitions should stay well under 100MB; use composite keys to split large ones.

16 Event-Driven Microservices Architecture (Choreographed Saga)

DEFINITION

- Services communicate purely through immutable events; there is no central orchestrator directing the flow.
- The system embraces eventual consistency, using compensating events for failure paths instead of 2PC.
- Idempotent consumers, deduplication tables, and DLQs make the choreography reliably repeatable.

HANDS-ON

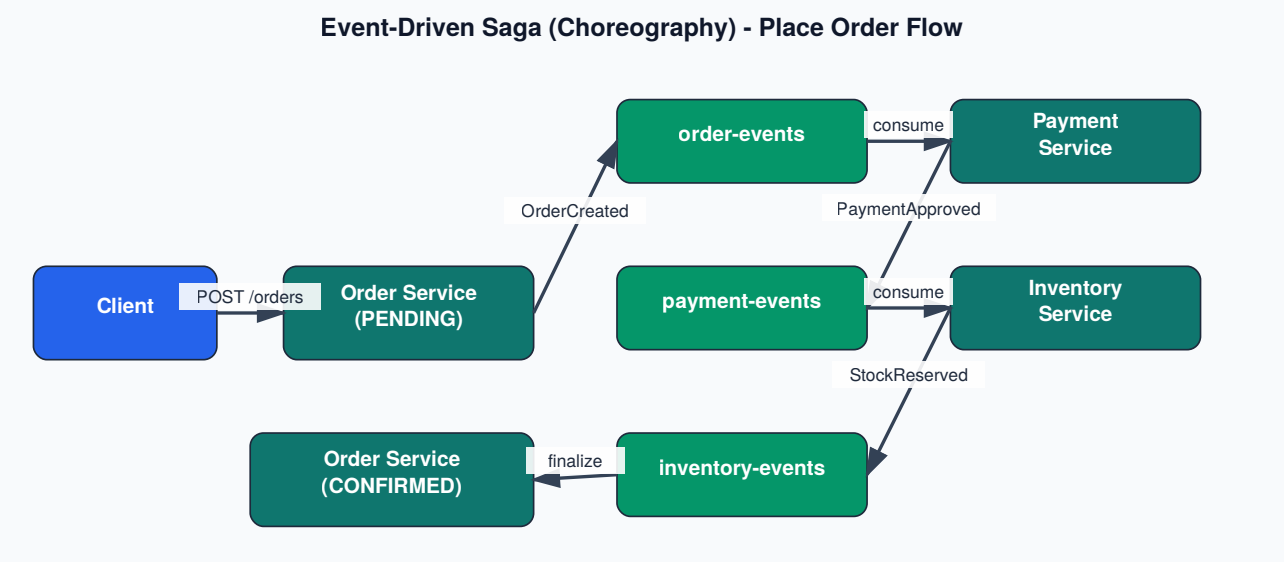
Idempotent Saga Finalization

```
@KafkaListener(topics = "inventory-events", groupId = "order-group")
public void consumeInventoryEvent(ConsumerRecord<String, OrderEvent> record, Acknowledgment ack) {
    String eventId = record.value().getEventId();
    if (dedupRepository.existsById(eventId)) { ack.acknowledge(); return; }  // idempotency guard
    orderRepository.updateStatus(record.value().getOrderId(), "CONFIRMED");
    dedupRepository.save(new ProcessedEvent(eventId, Instant.now()));
    ack.acknowledge();
}
```

REAL-TIME USAGE

Organization	How it is applied
Stripe	Payment events trigger fraud scoring and bank reconciliation asynchronously.
Airbnb	Booking events decouple host notification from the critical booking confirmation.
Robinhood	Trade events fan out to matching, risk clearing, and reporting independently.

ARCHITECTURAL FLOW DIAGRAM



1. Order Service saves PENDING and publishes OrderCreated without waiting for downstream services.
2. Payment Service consumes it, debits the account, and publishes PaymentApproved or PaymentFailed.
3. Inventory Service reacts to PaymentApproved, reserves stock, and publishes StockReserved.
4. Order Service consumes StockReserved (deduplicated) and finalizes the order as CONFIRMED.

Trade-offs / What this evolves into next

- UI must handle a visible 'processing' state during the eventual-consistency window.
- Compensation logic (cancel/refund events) must be designed for every failure branch.

17 Containers, Orchestration & CI/CD

DEFINITION

- Docker packages a service with all its dependencies for environment-consistent execution.
- Kubernetes automates deployment, scaling, self-healing, and rolling updates across a cluster.
- GitOps treats Git as the single source of truth; Argo CD reconciles cluster state automatically.

HANDS-ON

Multi-Stage Dockerfile + CI Pipeline Stage

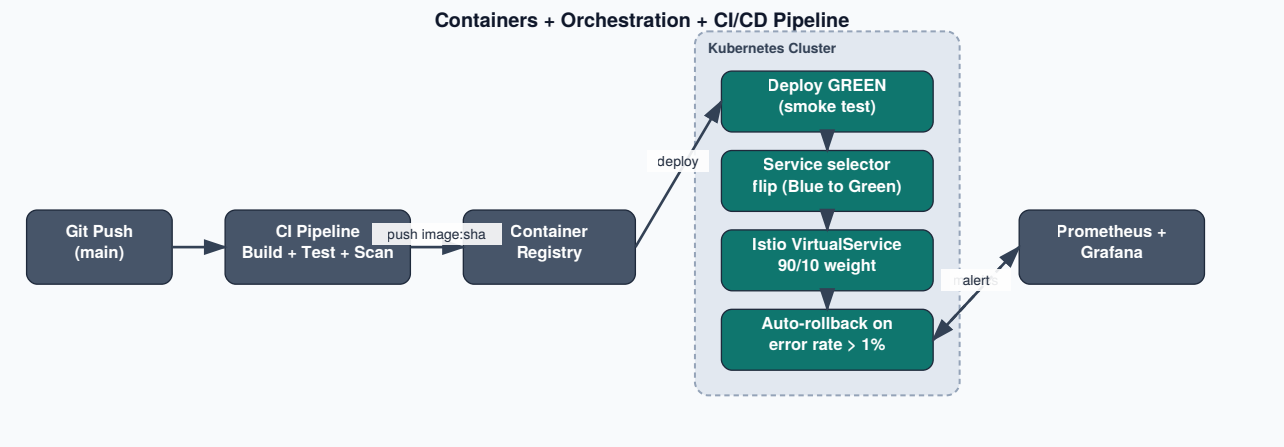
```
FROM maven:3.8-openjdk-17 AS build
COPY . .
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
COPY --from=build /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

REAL-TIME USAGE

Organization	How it is applied
Netflix (Spinnaker)	Bakes and tests a new build in isolation before flipping the load balancer.
Spotify (Argo CD)	GitOps continuously reconciles ~500 microservices against the declared Git state.
Etsy	300+ daily deploys using a Canary + Blue-Green hybrid with automated abort.

ARCHITECTURAL FLOW DIAGRAM



1. A merge to main triggers the CI pipeline: build, test, image scan, and push to the registry.
2. The new image deploys as GREEN and passes smoke tests before any live traffic reaches it.
3. The Service selector flips instantly from BLUE to GREEN (or a weighted Canary ramps gradually).
4. Prometheus-driven checks can auto-rollback the moment error rate crosses a safety threshold.

Trade-offs / What this evolves into next

- Requires disciplined use of immutable image tags and declarative manifests in Git.
- Observability must be wired in from day one to make rollback decisions automatic.